

Java Backend Architecture Interview Series

CHAPTER 1 · SUB-CHAPTER 1.5

Collections

Queue, Deque & BlockingQueue

PriorityQueue · ArrayDeque · LinkedBlockingQueue · ArrayBlockingQueue
SynchronousQueue · DelayQueue · PriorityBlockingQueue · LinkedTransferQueue

8 Documentation Sections

Producer-Consumer Patterns

Coding Exercises & Debug Drills

52+ Q&A (Standard + Advanced)

Heap Internals & Lock Analysis

Cheat Sheet Reference Table

Table of Contents

Section 1 — Queue & Deque Interface Hierarchy

Section 2 — PriorityQueue Deep Dive

Section 3 — ArrayDeque Deep Dive

Section 4 — LinkedBlockingQueue & ArrayBlockingQueue

Section 5 — Other BlockingQueue Implementations

Section 6 — Producer-Consumer Pattern

Section 7 — Deque as Stack vs Queue

Section 8 — Performance & Thread Safety Comparison

Q&A; — 52+ Questions — Standard (Q1–Q35) & Advanced (Q36–Q52)

Exercises — Parts A (Coding), B (Debug), C (MCQ)

Cheat Sheet — 18-Row Quick Reference Table

Section 1 — Queue & Deque Interface Hierarchy

Java's queue-related interfaces form a clean hierarchy in **java.util** and **java.util.concurrent**. Understanding which interface each class implements determines what blocking, ordering, and capacity guarantees are available.

Interface Hierarchy

```
java.util.Collection<E>
■ ■ java.util.Queue<E> // one-ended FIFO or priority
■ ■ java.util.Deque<E> // double-ended queue
■ ■ ■ java.util.ArrayDeque<E> // resizable circular array
■ ■ ■ java.util.LinkedList<E> // doubly linked (also List)
■ ■ java.util.concurrent.BlockingQueue<E>
■ ■ ■ LinkedBlockingQueue<E>
■ ■ ■ ArrayBlockingQueue<E>
■ ■ ■ PriorityBlockingQueue<E>
■ ■ ■ DelayQueue<E>
■ ■ ■ SynchronousQueue<E>
■ ■ ■ LinkedTransferQueue<E> // also implements TransferQueue
java.util.concurrent.BlockingDeque<E> // extends BlockingQueue + Deque
■ ■ ■ LinkedBlockingDeque<E>
```

Queue Method Pairs — Behavior on Empty / Full

Operation	Throws Exception	Returns Special Value	Blocks (BlockingQueue)	Times Out (BlockingQueue)
Insert	add(e) — throws <code>IllegalStateException</code> if full	offer(e) — returns false if full	put(e) — blocks until space	offer(e, t, unit) — waits t
Remove	remove() — throws <code>NoSuchElementException</code> if empty	poll() — returns null if empty	take() — blocks until element	poll(t, unit) — waits t
Examine	element() — throws <code>NoSuchElementException</code> if empty	peek() — returns null if empty	N/A	N/A

Deque — Additional Methods

Deque Method	Equivalent Queue/Stack	Description
addFirst(e) / offerFirst(e)	push(e) [stack]	Insert at head
addLast(e) / offerLast(e)	add(e) / offer(e) [queue]	Insert at tail
removeFirst() / pollFirst()	remove() / poll() [queue], pop() [stack]	Remove from head
removeLast() / pollLast()	— (queue has no equivalent)	Remove from tail
peekFirst() / getFirst()	peek() / element() [queue]	Examine head
peekLast() / getLast()	—	Examine tail
descendingIterator()	—	Iterate tail-to-head

KEY RULE: Always prefer the 'special value' forms (offer/poll/peek) over the 'exception' forms (add/remove/element) unless you explicitly want an exception to signal a programming error. For `BlockingQueue` use `put/take` when you want the calling thread to wait.

Section 2 — PriorityQueue Deep Dive

PriorityQueue<E> is an unbounded min-heap backed by an **Object[]** array. It does NOT guarantee FIFO ordering for equal-priority elements — only that the head is always the minimum (or maximum with a custom Comparator).

Complexity Summary

Operation	Time Complexity	Notes
offer(e) / add(e)	$O(\log n)$	Appends at end, sift-up to restore heap property
poll() / remove()	$O(\log n)$	Swaps root with last element, sift-down
peek() / element()	$O(1)$	Returns queue[0] directly
contains(Object)	$O(n)$	Linear scan — no positional index maintained
remove(Object)	$O(n)$	$O(n)$ to find + $O(\log n)$ sift — effectively $O(n)$
heapify (constructor)	$O(n)$	Floyd's bottom-up algorithm
size()	$O(1)$	Maintained as field

Heap Property & Internal Operations

- Min-heap invariant: $\text{queue}[\text{parent}] \leq \text{queue}[\text{child}]$ for all nodes
- Parent of index i : $(i-1)/2$ (integer division). Children of i : $2i+1$ (left), $2i+2$ (right)
- sift-up (after offer): compare new node with parent; swap upward while parent > child
- sift-down (after poll): swap root with last, remove last, push root down comparing with smaller child
- Natural ordering: elements must implement Comparable; null NOT allowed
- Custom ordering: pass Comparator at construction — overrides Comparable

Creating a Max-Heap

```
// Min-heap (default)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

// Max-heap using Comparator.reverseOrder()
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

// Custom comparator – sort tasks by priority (lower number = higher priority)
PriorityQueue<Task> taskQueue = new PriorityQueue<>(
    Comparator.comparingInt(Task::getPriority));

// Initialize from collection in  $O(n)$  using Floyd's heapify
List<Integer> list = Arrays.asList(5, 2, 8, 1, 9);
PriorityQueue<Integer> pq = new PriorityQueue<>(list); //  $O(n)$ 
```

Top-K Elements Pattern (Min-Heap of size K)

```
// Find top K largest elements from a stream –  $O(n \log K)$  time,  $O(K)$  space
public List<Integer> topK(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>(k);
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) {
            minHeap.poll(); // remove the smallest – keeps K largest
        }
    }
    return new ArrayList<>(minHeap);
}

// Why min-heap (NOT max-heap)?
// Max-heap needs to compare against all n elements to keep top-K
// Min-heap: peek() gives current K-th largest; poll if new element beats it
// Space:  $O(K)$  vs  $O(n)$  for sorting entire array
```

Merge K Sorted Lists Pattern

```
// Merge K sorted linked lists – O(N log K) where N = total elements
public ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> pq = new PriorityQueue<>(
        Comparator.comparingInt(n -> n.val));
    for (ListNode head : lists) {
        if (head != null) pq.offer(head);
    }
    ListNode dummy = new ListNode(0), cur = dummy;
    while (!pq.isEmpty()) {
        cur.next = pq.poll();
        cur = cur.next;
        if (cur.next != null) pq.offer(cur.next);
    }
    return dummy.next;
}
```

THREAD SAFETY: PriorityQueue is NOT thread-safe. Use PriorityBlockingQueue for concurrent access. The iterator of PriorityQueue does NOT guarantee sorted (heap) order — iterate by repeatedly calling poll() if you need sorted output.

Section 3 — ArrayDeque Deep Dive

ArrayDeque<E> is a resizable circular array implementing **Deque<E>**. It is the preferred general-purpose deque, stack, and FIFO queue in Java — faster than both **Stack** (synchronized) and **LinkedList** (pointer overhead, poor cache locality).

Complexity Summary

Operation	Time Complexity	Notes
addFirst / offerFirst	O(1) amortized	Wraps head pointer; may trigger doubleCapacity()
addLast / offerLast	O(1) amortized	Wraps tail pointer; may trigger doubleCapacity()
removeFirst / pollFirst	O(1)	Nulls out old head slot
removeLast / pollLast	O(1)	Nulls out old tail slot
peekFirst / peekLast	O(1)	Direct array access
contains(Object)	O(n)	Linear scan of circular buffer
size()	O(1)	Computed from head/tail indices
iterator()	O(n) total	Head-to-tail traversal in circular order

Circular Buffer Internals

- Backed by Object[] elements; capacity is always a power of 2 (bitmasking optimization)
- head = index of first element; tail = index past last element
- Empty when head == tail; full when (tail+1) & mask == head
- Wrap-around: new index = (index + 1) & mask (avoids modulo)
- doubleCapacity(): allocates 2x array, copies in two System.arraycopy calls
- Initial capacity: 16 elements (JDK 11+; was 8 in older JDKs)
- null elements NOT allowed — null is used as a sentinel for empty slots

Using ArrayDeque as a Stack (LIFO)

```
Deque<String> stack = new ArrayDeque<>();
// Push to top of stack
stack.push("first"); // addFirst(e)
stack.push("second"); // addFirst(e) – "second" is now on top
// Peek at top without removing
String top = stack.peek(); // peekFirst() – returns "second"
// Pop from top
String popped = stack.pop(); // removeFirst() – removes "second"
// Stack-style DFS traversal
Deque<Node> dfsStack = new ArrayDeque<>();
dfsStack.push(root);
while (!dfsStack.isEmpty()) {
    Node node = dfsStack.pop();
    process(node);
    if (node.right != null) dfsStack.push(node.right);
    if (node.left != null) dfsStack.push(node.left);
}
```

Using ArrayDeque as a Queue (FIFO)

```
Queue<String> queue = new ArrayDeque<>();
// Enqueue at tail
queue.offer("first"); // addLast(e)
queue.offer("second"); // addLast(e)
// Peek at head
String head = queue.peek(); // peekFirst() – returns "first"
```

```
// Dequeue from head
String dequeued = queue.poll(); // removeFirst() – removes "first"

// BFS traversal
Queue<Node> bfsQueue = new ArrayDeque<>();
bfsQueue.offer(root);
while (!bfsQueue.isEmpty()) {
    Node node = bfsQueue.poll();
    process(node);
    if (node.left != null) bfsQueue.offer(node.left);
    if (node.right != null) bfsQueue.offer(node.right);
}
```

Using ArrayDeque as a Full Deque

```
Deque<Integer> deque = new ArrayDeque<>();
deque.offerFirst(1); // [1]
deque.offerLast(2); // [1, 2]
deque.offerFirst(0); // [0, 1, 2]
int front = deque.peekFirst(); // 0
int back = deque.peekLast(); // 2
deque.pollFirst(); // removes 0 – [1, 2]
deque.pollLast(); // removes 2 – [1]

// Monotonic deque – sliding window max pattern
Deque<Integer> mono = new ArrayDeque<>();
// Maintain decreasing order; front always has the window's maximum
```

PREFER ArrayDeque OVER: (1) Stack class — Stack extends Vector which is fully synchronized, making even single-threaded use slow. (2) LinkedList as queue — LinkedList has poor cache locality due to pointer chasing and higher memory overhead (24 bytes per node vs ~4-8 bytes in array).

Section 4 — LinkedBlockingQueue & ArrayBlockingQueue

Both implement **BlockingQueue<E>** and are designed for the classic **producer-consumer pattern**. The critical difference lies in their internal locking strategy.

LinkedBlockingQueue

- Optionally bounded — default capacity is Integer.MAX_VALUE (effectively unbounded)
- DUAL LOCK design: putLock (guards tail/put) + takeLock (guards head/take)
- Allows simultaneous put() and take() — higher throughput under contention
- AtomicInteger count coordinates both locks without requiring a single global lock
- Linked nodes: each node holds item + next reference — higher GC pressure
- Thread-safe size() via AtomicInteger count field

ArrayBlockingQueue

- Bounded — capacity MUST be specified at construction; cannot grow
- SINGLE ReentrantLock for both put and take — put and take contend on same lock
- Optional fairness: new ArrayBlockingQueue<>(capacity, true) — FIFO lock ordering
- Fairness degrades throughput but prevents thread starvation
- Backed by circular Object[] — no per-element node allocation, lower GC pressure
- Two Condition objects: notEmpty (for waiting takes) + notFull (for waiting puts)

Blocking Operations Reference

Method	Behavior When Full (put)	Behavior When Empty (take)	Use Case
put(e)	Blocks indefinitely	N/A (producer method)	Unbounded wait for space
offer(e)	Returns false immediately	N/A	Non-blocking try-insert
offer(e,t,unit)	Waits up to timeout	N/A	Timed try-insert
take()	N/A	Blocks indefinitely	Unbounded wait for element
poll()	N/A	Returns null immediately	Non-blocking try-remove
poll(t,unit)	N/A	Waits up to timeout	Timed try-remove
drainTo(col)	N/A	Returns 0 if empty	Bulk atomic drain
drainTo(col,n)	N/A	Drains up to n elements	Partial atomic drain

LinkedBlockingQueue vs ArrayBlockingQueue — Comparison

Feature	LinkedBlockingQueue	ArrayBlockingQueue
Capacity	Optional (default: MAX_VALUE)	Mandatory bounded
Locking	Dual lock (putLock + takeLock)	Single ReentrantLock
Concurrent put+take	Yes (dual lock allows)	No (same lock)
Fairness option	No	Yes (constructor flag)
Memory	Higher (node objects)	Lower (array)
GC pressure	Higher	Lower
Throughput (high)	Generally higher	Lower due to contention
Null allowed	No	No
Ordering	FIFO	FIFO

```
// LinkedBlockingQueue – unbounded (use with care – memory risk!)
BlockingQueue<Task> unbounded = new LinkedBlockingQueue<>();

// LinkedBlockingQueue – bounded (recommended)
BlockingQueue<Task> bounded = new LinkedBlockingQueue<>(1000);
```



```
// ArrayBlockingQueue – bounded, fair
BlockingQueue<Task> fair = new ArrayBlockingQueue<>(1000, true);
// Producer: blocks if queue full
bounded.put(new Task("job-1"));
// Consumer: blocks if queue empty
Task task = bounded.take();
// Timed operations
boolean inserted = bounded.offer(new Task("job-2"), 500, TimeUnit.MILLISECONDS);
Task t = bounded.poll(500, TimeUnit.MILLISECONDS); // null if timeout
// Drain all pending tasks for batch processing
List<Task> batch = new ArrayList<>();
bounded.drainTo(batch, 100); // drain up to 100
```

Section 5 — Other BlockingQueue Implementations

PriorityBlockingQueue

- Unbounded — put() NEVER blocks (capacity grows like PriorityQueue)
- take() blocks when queue is empty
- Thread-safe priority queue using a single ReentrantLock
- Natural ordering (Comparable) or Comparator; no null elements
- Iterator does NOT guarantee sorted order (same as PriorityQueue)
- Use case: multi-threaded task scheduling by priority

```
PriorityBlockingQueue<Task> pbq =
new PriorityBlockingQueue<>(11, Comparator.comparingInt(Task::getPriority));
pbq.put(new Task(3, "low")); // never blocks
pbq.put(new Task(1, "urgent")); // never blocks – unbounded
Task highest = pbq.take(); // blocks if empty; returns highest-priority task
```

DelayQueue

- Elements must implement the Delayed interface (extends Comparable)
- take() blocks until the element at the head has an expired delay (getDelay() <= 0)
- Elements are ordered by remaining delay — minimum delay at head
- Internally: PriorityQueue + ReentrantLock + Condition
- Leader-follower pattern: only one thread (leader) waits with timed park; others wait indefinitely
- Use cases: session expiry, cache TTL eviction, retry with backoff, scheduled tasks

```
class DelayedTask implements Delayed {
private final String name;
private final long triggerTime; // System.nanoTime() + delay
DelayedTask(String name, long delayMs) {
this.name = name;
this.triggerTime = System.nanoTime() + TimeUnit.MILLISECONDS.toNanos(delayMs);
}
@Override
public long getDelay(TimeUnit unit) {
return unit.convert(triggerTime - System.nanoTime(), TimeUnit.NANOSECONDS);
}
@Override
public int compareTo(Delayed other) {
return Long.compare(triggerTime, ((DelayedTask)other).triggerTime);
}
}

DelayQueue<DelayedTask> dq = new DelayQueue<>();
dq.put(new DelayedTask("retry-job", 5000)); // fires after 5 seconds
DelayedTask task = dq.take(); // blocks until delay expires
```

SynchronousQueue

- Zero internal storage — NO buffering at all
- Each put() must wait for a matching take() — a direct hand-off (rendezvous)
- take() must wait for a matching put()
- Two modes: FIFO (fair=true, queue-based) or LIFO (fair=false, stack-based, default)
- Both modes are lock-free using CAS (compare-and-swap) operations
- Used internally by Executors.newCachedThreadPool() — threads hand tasks directly
- Throughput can be very high for matched producer-consumer pairs

```
SynchronousQueue<String> sq = new SynchronousQueue<>();
// Producer thread – blocks until consumer calls take()
new Thread(() -> {
try { sq.put("data"); System.out.println("Handed off!"); }
catch (InterruptedException e) { Thread.currentThread().interrupt(); }
}).start();
// Consumer thread – unblocks the producer
new Thread(() -> {
try { String data = sq.take(); System.out.println("Received: " + data); }
catch (InterruptedException e) { Thread.currentThread().interrupt(); }
}).start();
```

LinkedTransferQueue

- Implements TransferQueue — extends BlockingQueue with transfer() method
- transfer(e): blocks until a consumer directly receives the element (like SynchronousQueue.put())
- tryTransfer(e): returns false immediately if no consumer is waiting (non-blocking)
- Combines: SynchronousQueue-style direct handoff + LinkedBlockingQueue-style buffering
- Unbounded, non-blocking enqueue if no consumer waiting — falls back to buffering
- Lock-free implementation using CAS — generally lowest latency of all BlockingQueues
- Theoretically the best general BlockingQueue since JDK 7

Section 6 — Producer-Consumer Pattern

The producer-consumer pattern decouples work generation from work execution. A **bounded BlockingQueue** acts as the buffer and naturally applies backpressure when producers outpace consumers.

Classic Producer-Consumer with Poison Pill Shutdown

```
public class ProducerConsumerDemo {
    private static final int NUM_PRODUCERS = 2;
    private static final int NUM_CONSUMERS = 4;
    private static final Task POISON_PILL = new Task("STOP");
    private final BlockingQueue<Task> queue = new LinkedBlockingQueue<>(500);

    class Producer implements Runnable {
        @Override public void run() {
            try {
                for (int i = 0; i < 100; i++) {
                    queue.put(new Task("task-" + i)); // blocks if queue full
                }
                // One poison pill per consumer
                for (int i = 0; i < NUM_CONSUMERS; i++) {
                    queue.put(POISON_PILL);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }

    class Consumer implements Runnable {
        @Override public void run() {
            try {
                while (true) {
                    Task task = queue.take(); // blocks if queue empty
                    if (task == POISON_PILL) return; // shutdown signal
                    process(task);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }

    public void start() throws InterruptedException {
        ExecutorService exec = Executors.newFixedThreadPool(
            NUM_PRODUCERS + NUM_CONSUMERS);
        for (int i = 0; i < NUM_PRODUCERS; i++) exec.submit(new Producer());
        for (int i = 0; i < NUM_CONSUMERS; i++) exec.submit(new Consumer());
        exec.shutdown();
        exec.awaitTermination(60, TimeUnit.SECONDS);
    }
}
```

Backpressure with Bounded Queue

- A bounded BlockingQueue naturally throttles producers: when full, put() blocks
- This prevents unbounded memory growth — the queue capacity acts as a flow-control valve
- Timed offer() lets producers detect congestion and implement drop/retry strategies
- Monitoring: queue.size() / queue.remainingCapacity() for health metrics

```
// Producer with timeout-based backpressure strategy
```

```
boolean offered = queue.offer(task, 100, TimeUnit.MILLISECONDS);
if (!offered) {
    metrics.increment("queue.full.drops");
    // Strategy: drop, retry, or route to overflow handler
}
// Bulk drain – consumer processes in batches for throughput
List<Task> batch = new ArrayList<>(50);
queue.drainTo(batch, 50); // atomically grab up to 50 tasks
processBatch(batch);
```

MODERN ALTERNATIVE: Reactive Streams (Project Reactor / RxJava) implement backpressure via the `Subscriber.request(n)` protocol — consumers signal how many items they can handle. This avoids blocking threads entirely. For new systems handling millions of events, consider Flux/Mono over raw `BlockingQueue`.

Section 7 — Deque as Stack vs Queue

Java's **Stack** class is a legacy artifact extending **Vector** (fully synchronized). The Java documentation itself recommends using **Deque** instead. The choice between **ArrayDeque** and **LinkedList** as the Deque implementation matters for performance.

Stack Class — Why to Avoid

- Stack extends **Vector** — every method (push, pop, peek, empty, search) is synchronized
- Single-threaded use still pays the synchronization overhead on every operation
- Inherits all List methods from **Vector** — Stack 'is-a' List, which violates encapsulation
- Allows random access and insertion anywhere — not a true stack contract
- Fix: `Deque stack = new ArrayDeque<>()` — no synchronization, pure stack behavior

ArrayDeque vs LinkedList as Deque

Feature	ArrayDeque	LinkedList
Data structure	Circular Object[] array	Doubly-linked list
Memory per element	~4-8 bytes (array slot)	~24-32 bytes (Node object with 2 pointers)
Cache locality	Excellent (contiguous memory)	Poor (pointer chasing)
null elements	NOT allowed (sentinel use)	Allowed
Random access	Not efficient	Not efficient
Add/remove at ends	O(1) amortized	O(1) guaranteed
Add/remove at middle	O(n)	O(n) find + O(1) remove
Also implements	Deque only	Deque + List + Queue
Thread safety	Not thread-safe	Not thread-safe
Preferred for stack	YES — faster, less memory	No — unnecessary overhead
Preferred for queue	YES — better locality	No (unless null needed)

Method Mapping Summary

Use Case	Method to Use on ArrayDeque	Equivalent Meaning
Stack push	<code>push(e)</code> or <code>addFirst(e)</code>	Insert at head (top of stack)
Stack pop	<code>pop()</code> or <code>removeFirst()</code>	Remove from head
Stack peek	<code>peek()</code> or <code>peekFirst()</code>	Read head without removing
Queue enqueue	<code>offer(e)</code> or <code>addLast(e)</code>	Insert at tail
Queue dequeue	<code>poll()</code> or <code>removeFirst()</code>	Remove from head
Queue peek	<code>peek()</code> or <code>peekFirst()</code>	Read head without removing
Deque front insert	<code>offerFirst(e)</code>	Insert at head
Deque rear insert	<code>offerLast(e)</code>	Insert at tail
Deque front remove	<code>pollFirst()</code>	Remove from head
Deque rear remove	<code>pollLast()</code>	Remove from tail

Section 8 — Performance & Thread Safety Comparison

Class	Thread-Safe	Bounded	Null OK	Blocking	Ordering	Head/Tail O
PriorityQueue	No	No (unbounded)	No	No	Priority (min-heap)	$O(\log n)$ / $O(1)$
ArrayDeque	No	No (resizable)	No	No	FIFO or LIFO	$O(1)$ amortized
LinkedList	No	No (unbounded)	Yes	No	FIFO or LIFO	$O(1)$
Stack (legacy)	Yes (sync)	No	Yes	No	LIFO	$O(1)$ sync
LinkedBlockingQueue	Yes	Optional	No	Yes	FIFO	$O(1)$
ArrayBlockingQueue	Yes	Yes (mandatory)	No	Yes	FIFO	$O(1)$
PriorityBlockingQueue	Yes	No (unbounded)	No	take only	Priority	$O(\log n)$ / $O(1)$
DelayQueue	Yes	No (unbounded)	No	Yes (delay)	By expiry time	$O(\log n)$ / $O(1)$
SynchronousQueue	Yes	Zero capacity	No	Yes (always)	N/A (no storage)	$O(1)$
LinkedTransferQueue	Yes	No (unbounded)	No	Conditional	FIFO	$O(1)$
LinkedBlockingDeque	Yes	Optional	No	Yes	FIFO/LIFO	$O(1)$
ConcurrentLinkedQueue	Yes (CAS)	No (unbounded)	No	No	FIFO	$O(1)$

CHOOSING A QUEUE: (1) Single-threaded stack/queue → ArrayDeque. (2) Single-threaded priority → PriorityQueue. (3) Concurrent producer-consumer (bounded) → ArrayBlockingQueue (low GC) or LinkedBlockingQueue (higher throughput). (4) Priority with concurrency → PriorityBlockingQueue. (5) Scheduled/delayed tasks → DelayQueue. (6) Direct handoff (no buffer) → SynchronousQueue. (7) Highest throughput blocking queue → LinkedTransferQueue.

Q&A; — Standard Questions (Q1–Q35)

Q1. What is the difference between `offer()` and `add()` in the Queue interface?

Both attempt to insert an element. `add()` throws `IllegalStateException` if the queue is capacity-restricted and full. `offer()` returns false instead of throwing. For `BlockingQueue`, `add()` also throws; `offer()` returns false immediately. Prefer `offer()` in general code to avoid exception-based flow control.

Q2. What happens when you call `poll()` on an empty queue versus `remove()`?

`poll()` returns null when the queue is empty — a safe 'no element' sentinel. `remove()` throws `NoSuchElementException`. Use `poll()` with a null check in normal code; use `remove()` only when an empty queue represents a programming error.

Q3. How does `PriorityQueue` maintain heap order internally?

`PriorityQueue` uses a binary min-heap stored in an `Object[]` array. On `offer()`, the element is appended at the end (index size) then sifted up: repeatedly compared with its parent at $(i-1)/2$ and swapped if smaller, until the heap property is restored. On `poll()`, the root is removed, the last element is placed at the root, then sifted down by comparing with the smaller child until the heap property is restored.

Q4. Why does `PriorityQueue`'s iterator not guarantee sorted order?

The `PriorityQueue` iterator simply traverses the underlying array in index order (0, 1, 2, ...). A binary heap stores the minimum at index 0, but the rest of the array has no guaranteed sorted order — only the parent $\leq$ children invariant. To consume in sorted order, repeatedly call `poll()` in a loop.

Q5. When would you use `ArrayDeque` as a stack instead of the `Stack` class?

Always prefer `ArrayDeque`. The `Stack` class extends `Vector`, so every operation (`push`, `pop`, `peek`, `empty`) is synchronized — pointless overhead in single-threaded code. `ArrayDeque` is unsynchronized, has better cache locality (contiguous array), and does not expose random-access `List` methods. Declare as `Deque stack = new ArrayDeque<>()` for the cleanest interface.

Q6. What is the key difference between `LinkedBlockingQueue` and `ArrayBlockingQueue`?

`LinkedBlockingQueue` uses two separate locks (`putLock` for tail, `takeLock` for head), allowing concurrent `put()` and `take()` operations. `ArrayBlockingQueue` uses a single `ReentrantLock` for both, so `put` and `take` contend. `LinkedBlockingQueue` is optionally bounded (default `Integer.MAX_VALUE`); `ArrayBlockingQueue` requires a capacity at construction. `ArrayBlockingQueue` has an optional fairness mode; `LinkedBlockingQueue` does not.

Q7. What is the capacity of a `LinkedBlockingQueue` if none is specified?

`Integer.MAX_VALUE` (2,147,483,647). This makes it effectively unbounded from a practical standpoint, but it is NOT truly unbounded — it's an extremely large capacity that could cause `OutOfMemoryError` if producers consistently outpace consumers. Always specify a capacity in production code.

Q8. How does `SynchronousQueue` work and what makes it special?

`SynchronousQueue` has zero internal storage. Each `put()` must wait for a corresponding `take()`, and each `take()` must wait for a `put()` — a direct rendezvous hand-off between threads. It supports two modes: unfair (default, LIFO stack-based) and fair (FIFO queue-based). Both modes are lock-free using CAS. It is used in `Executors.newCachedThreadPool()` to hand tasks directly to idle threads.

Q9. What is the `Delayed` interface and how does `DelayQueue` use it?

`Delayed` extends `Comparable` and requires `getDelay(TimeUnit unit)` to return remaining delay time. `DelayQueue` internally stores elements in a `PriorityQueue` ordered by delay (minimum delay at head). `take()` checks `getDelay()` on the head element: if ≤ 0 , the element has expired and is returned; otherwise the thread waits (with timed park) for the delay to expire.

Q10. How do you implement a producer-consumer pattern with BlockingQueue?

Create a bounded BlockingQueue (e.g., `LinkedBlockingQueue(500)`). Producers call `put()` to insert tasks (blocks if full — natural backpressure). Consumers call `take()` to retrieve tasks (blocks if empty). Use a poison pill (a sentinel value) to signal consumers to terminate — put one pill per consumer thread. Wrap blocking calls in try-catch for `InterruptedException` and restore the interrupt flag.

Q11. Why does `put()` block but `offer()` does not always block?

`put()` is a BlockingQueue-specific method that semantically means 'insert, waiting as long as necessary for space.' `offer()` is a Queue method returning boolean — non-blocking. The timed variant `offer(e, t, unit)` will block up to the specified timeout. This distinction lets you choose between hard blocking (`put`), soft timeout (timed `offer`), and immediate failure (`offer`).

Q12. How do you atomically drain all elements from a BlockingQueue?

Use `drainTo(Collection c)` which atomically removes all available elements and adds them to `c`. The variant `drainTo(c, maxElements)` drains up to `maxElements`. Both return the number of elements drained. This is more efficient than looping `poll()` because it holds the lock for fewer acquisitions (one vs `n`).

Q13. What happens when you call `put()` on an unbounded PriorityBlockingQueue?

`put()` never blocks on `PriorityBlockingQueue` because it is unbounded. The element is simply added to the internal priority queue under a `ReentrantLock` and the capacity never exhausts (it grows dynamically). Only `take()` blocks — when the queue is empty. This means producers can outpace consumers indefinitely, consuming unbounded memory. Monitor queue size in production.

Q14. Can you use ArrayDeque as a min-heap? If not, what should you use instead?

No. `ArrayDeque` is a double-ended queue — it maintains insertion order (circular buffer) and has no concept of priority ordering. It provides $O(1)$ operations at both ends but cannot be used as a heap. For a min-heap, use `PriorityQueue` (single-threaded) or `PriorityBlockingQueue` (concurrent). A heap requires sift-up/sift-down on insertion/removal, which `ArrayDeque`'s circular buffer does not do.

Q15. What is the difference between `peek()` and `element()` on Queue?

Both retrieve (without removing) the head element. `peek()` returns null if the queue is empty. `element()` throws `NoSuchElementException` if the queue is empty. Use `peek()` when an empty queue is a normal condition; use `element()` when an empty queue represents a bug that should fail loudly.

Q16. When would you use SynchronousQueue in a thread pool?

`SynchronousQueue` is ideal when you want tasks handed directly to idle threads with no queuing — i.e., if no thread is available the submission fails or a new thread is created. `Executors.newCachedThreadPool()` uses `SynchronousQueue`: if no idle thread is available for a direct handoff, a new thread is spawned. This avoids task queuing overhead for short-lived, CPU-intensive tasks.

Q17. How does `Executors.newCachedThreadPool()` use SynchronousQueue?

`newCachedThreadPool` creates a `ThreadPoolExecutor` with `corePoolSize=0`, `maximumPoolSize=Integer.MAX_VALUE`, `keepAliveTime=60s`, and a `SynchronousQueue` as the work queue. When a task is submitted, `offer()` attempts a direct handoff to a waiting thread. If no thread is waiting (`offer` returns false), the executor creates a new thread. Idle threads wait up to 60s then terminate, making the pool elastic.

Q18. What is a DelayQueue used for in practice?

`DelayQueue` is used for time-based scheduling: (1) Session/cache expiry — sessions expire after idle timeout. (2) Retry with backoff — failed tasks re-enqueued with increasing delay. (3) Rate limiting — tokens added to queue with fixed delay. (4) Scheduled task execution — tasks fire at a specific future time. Java's `ScheduledThreadPoolExecutor` uses a similar priority-delay queue internally.

Q19. What is the initial capacity and growth strategy of ArrayDeque?

Initial capacity is 16 elements (power of 2; was 8 in older JDKs before JDK 11). When the circular buffer becomes full ($\text{head} == (\text{tail} + 1) \& \text{mask}$), `doubleCapacity()` is called. It allocates a new array of 2x size and copies elements in two `System.arraycopy` calls — one for elements from head to end of old array, one for elements from start to tail. Capacity is always kept as a power of 2 for efficient bitmasking.

Q20. How would you implement a simple task scheduler using PriorityQueue?

Define `ScheduledTask` with a `triggerTime` field; implement `Comparable` comparing by `triggerTime`. A scheduler thread loop: peek at head task; if $\text{triggerTime} \leq \text{now}$, `poll()` and execute in a thread pool; else `sleep(triggerTime - now)`. On new task submission, `offer()` to queue and wake the scheduler thread (using `Object.notify()` or a `Condition`). For thread safety use `PriorityBlockingQueue` instead.

Q21. What is the difference between Queue and Deque interfaces?

`Queue` is a single-ended queue — elements enter at the tail and exit at the head (FIFO by default). `Deque` (Double-Ended Queue) extends `Queue` and allows insertion and removal at BOTH ends. `Deque` can emulate a stack (LIFO) by using the head end, or a queue (FIFO) using head+tail. Key additions: `offerFirst/offerLast`, `pollFirst/pollLast`, `peekFirst/peekLast`.

Q22. Can you store null in a PriorityQueue or ArrayDeque?

No for both. `PriorityQueue` rejects null because null cannot be compared (`Comparable.compareTo(null)` throws `NullPointerException`). `ArrayDeque` rejects null because it uses null as an internal sentinel to detect empty slots in the circular buffer. `LinkedList`, by contrast, allows null elements.

Q23. What is ConcurrentLinkedQueue and when do you use it instead of LinkedBlockingQueue?

`ConcurrentLinkedQueue` is a non-blocking, lock-free FIFO queue using the Michael-Scott algorithm (CAS). It never blocks — `offer()` and `poll()` return immediately. Use it when producers and consumers are always active and you want to avoid thread blocking overhead. Drawback: `size()` is $O(n)$ (full traversal). Use `LinkedBlockingQueue` when you need blocking semantics (e.g., consumers should wait for work).

Q24. How does BlockingDeque differ from BlockingQueue?

`BlockingDeque` extends both `BlockingQueue` and `Deque` — it adds blocking operations at both ends. It provides: `putFirst/putLast` (blocking inserts), `takeFirst/takeLast` (blocking removes), `offerFirst/offerLast` with timeout, `pollFirst/pollLast` with timeout. `LinkedBlockingDeque` is the standard implementation. Useful for work-stealing algorithms where workers steal from the tail of other workers' dequeues.

Q25. What guarantees does ArrayBlockingQueue's fairness mode provide?

When constructed with `ArrayBlockingQueue(capacity, true)`, the internal `ReentrantLock` is created in fair mode (`ReentrantLock(true)`). This ensures threads waiting to `put()` or `take()` are served in FIFO order — the longest-waiting thread gets access first. This prevents starvation but significantly reduces throughput because fair locks prevent barging (non-queued threads jumping ahead).

Q26. What does the transfer() method in LinkedTransferQueue do differently from put()?

`put()` inserts the element and returns immediately if the queue is bounded and has space. `transfer(e)` inserts but WAITS until a consumer actually receives the element — it blocks until some thread calls `poll()` or `take()` and retrieves this specific element. This guarantees the element was consumed before returning, enabling synchronous handoff semantics like `SynchronousQueue` but with `LinkedBlockingQueue`-style buffering when no consumer is waiting.

Q27. How do you implement a monotonic deque for sliding window maximum?

Use `ArrayDeque` storing array indices. Invariant: deque front holds index of current window max. For each new element: (1) Remove from front any index outside the window ($\text{index} \leq i - k$). (2) Remove from back any index whose value is $\leq$ current element (maintaining decreasing order). (3) Add current index to back. (4) Front of deque is always the max for the current window. Time: $O(n)$, Space: $O(k)$.

Q28. Why is LinkedList generally not recommended as a Queue implementation in performance-critical code?

LinkedList allocates a Node object for each element (containing item, prev, next pointers — ~24-32 bytes per element). This causes: (1) Higher GC pressure from node allocation/collection. (2) Poor cache locality — node objects scattered in heap, causing cache misses during traversal. ArrayDeque uses a contiguous array — sequential access hits cache lines efficiently. In benchmarks, ArrayDeque is typically 2-5x faster than LinkedList for queue operations.

Q29. What is the 'poison pill' pattern in producer-consumer, and what are its limitations?

A poison pill is a special sentinel object placed in the queue to signal consumers to terminate. Each consumer removes one pill and exits. Limitations: (1) Number of pills must match consumer count — difficult if consumer count changes dynamically. (2) If a consumer dies unexpectedly, the pill sits in the queue unconsumed. (3) Works poorly with PriorityQueue where pill might not be dequeued first. Alternative: use a volatile boolean flag + interrupt-based shutdown.

Q30. How does PriorityQueue differ from a sorted collection like TreeSet?

PriorityQueue allows duplicates; TreeSet does not. PriorityQueue only guarantees the minimum is at the head ($O(1)$ peek); full sorted traversal requires $O(n \log n)$ via repeated poll(). TreeSet is always fully sorted but iteration is $O(n)$. PriorityQueue operations: $O(\log n)$ insert/remove; TreeSet: $O(\log n)$ insert/remove. PriorityQueue is more memory-efficient (array vs tree nodes).

Q31. What is the difference between remainingCapacity() and size() in BlockingQueue?

size() returns the number of elements currently in the queue. remainingCapacity() returns how many more elements can be added before the queue is full (capacity - size). For an unbounded queue like LinkedBlockingQueue with default capacity, remainingCapacity() returns Integer.MAX_VALUE - size. Both are approximate in concurrent use — by the time you act on them, the queue may have changed.

Q32. Can two threads simultaneously call put() and take() on the same LinkedBlockingQueue?

Yes — this is the key advantage of LinkedBlockingQueue's dual-lock design. putLock guards the tail (used by put/offer), while takeLock guards the head (used by take/poll). These are independent ReentrantLocks, so a producer holding putLock and a consumer holding takeLock can execute concurrently. An AtomicInteger count field coordinates between the two locks without a shared lock.

Q33. What happens when you iterate over a BlockingQueue while other threads modify it?

BlockingQueue iterators are weakly consistent — they do not throw ConcurrentModificationException. The iterator reflects the state at or after the time of iterator creation. Elements added after the iterator was created may or may not be visible. Elements removed may or may not appear. This is safe to use for monitoring/logging but should not be relied on for correctness in concurrent logic.

Q34. What is the difference between BlockingQueue and TransferQueue?

BlockingQueue adds blocking put/take semantics to Queue. TransferQueue extends BlockingQueue with transfer(e) — which blocks until a consumer directly receives the element — and tryTransfer(e) — which succeeds only if a consumer is already waiting. TransferQueue can report getWaitingConsumerCount(). LinkedTransferQueue is the only JDK implementation of TransferQueue.

Q35. How would you choose between PriorityBlockingQueue and a regular PriorityQueue with synchronization?

PriorityBlockingQueue is preferred for concurrent use because: (1) It provides blocking take() — threads can wait efficiently for elements instead of spinning. (2) Its lock-based design prevents race conditions in the heap operations. Manually synchronizing PriorityQueue (e.g., with Collections.synchronizedQueue) does not add blocking semantics — threads would need to busy-wait or use external Conditions. PriorityBlockingQueue handles all of this correctly.

Q&A; — Advanced Questions (Q36–Q52)

[ADV] Q36. Explain PriorityQueue's sift-up algorithm. How does insertion at the end then sift-up maintain the heap property? Prove $O(\log n)$.

On `offer(e)`: the element is placed at index $k = \text{size}$ (the next available leaf position). `siftUp(k, e)`: compare e with parent at $(k-1)/2$. If $e < \text{parent}$, swap and set $k = (k-1)/2$; repeat. This traverses at most the height of the binary heap = $\text{floor}(\log_2(n+1))$. At each level, $O(1)$ comparison and possible swap. Total: $O(\log n)$ comparisons. The heap property is restored because: after sifting, $e \geq$ its new parent (loop terminated) and $e \leq$ its children (it was placed at what was a valid leaf, so no children yet OR was swapped from a position where children are already $\geq$ the displaced parent).

[ADV] Q37. How does PriorityQueue's `heapify()` constructor achieve $O(n)$ rather than $O(n \log n)$ for repeated insertions? Explain Floyd's algorithm.

Floyd's algorithm builds a heap bottom-up: start from the last non-leaf (index $n/2 - 1$) down to 0, calling `siftDown()` on each. The key insight: the cost of `siftDown` at height h is $O(h)$, and there are approximately $n/2^{h+1}$ nodes at height h . Total cost = sum over h from 0 to $\log(n)$ of $n/2^{h+1} * h = n * \sum(h/2^h)$ which converges to 2 — giving $O(n)$. Repeated `offer()` is $O(n \log n)$ because every insertion does $O(\log n)$ sift-up. Floyd's avoids building up from empty; instead it repairs existing structure bottom-up.

[ADV] Q38. Explain LinkedBlockingQueue's dual-lock design. How does AtomicInteger count coordinate between `putLock` and `takeLock`?

`LinkedBlockingQueue` has two `ReentrantLock`s: `putLock` (with `notFull` Condition) and `takeLock` (with `notEmpty` Condition). Producers acquire `putLock` to modify the tail; consumers acquire `takeLock` to modify the head. `AtomicInteger` count is visible to both without locking. After a `put`: `count.getAndIncrement()` — if result was 0 (queue was empty), signal `notEmpty` under `takeLock`. After a `take`: if `count.getAndDecrement()` was capacity (queue was full), signal `notFull` under `putLock`. This coordination allows concurrent head/tail modifications while signaling conditions across lock boundaries.

[ADV] Q39. Why does ArrayBlockingQueue's fair mode (`ReentrantLock(true)`) cause performance degradation, and when is fairness worth using?

A fair `ReentrantLock` uses a CLH queue (FIFO ordering). When a thread releases the lock, the next thread in the queue is unparked — this involves OS scheduling overhead. In unfair mode, a newly arrived thread can acquire the lock immediately (barging) without unparking, which is faster if the OS context switch is more expensive than the contention window. Benchmarks show fair mode can reduce throughput by 3-10x under high contention. Use fairness when: thread starvation is observed, all threads have equal priority and wait times matter for correctness.

[ADV] Q40. SynchronousQueue has two modes — stack-based (LIFO) for unfair, queue-based (FIFO) for fair. How are both implemented lock-free using CAS?

`SynchronousQueue` uses a `Transferer` inner class with two implementations: `TransferStack` (unfair/LIFO) and `TransferQueue` (fair/FIFO). Both maintain a linked list of waiting nodes using CAS to atomically update head/top references. A node represents either a data item (`put`) or a request (`take`). When a complementary operation arrives, it finds the waiting node via CAS match and completes the transfer without locks. ABA problem is avoided because nodes are never reused. `TransferStack` uses a stack where the most recent waiter is served first (LIFO). `TransferQueue` uses a FIFO linked list where the oldest waiter is served first.

[ADV] Q41. How does SynchronousQueue's transfer-based handoff avoid allocating a buffer, enabling direct producer-to-consumer handoff?

When a producer calls `put(e)`, a `SNode` is created with the data item and pushed onto the stack/queue. When a consumer calls `take()`, it finds the producer's `SNode`, writes its own reference into the node as a 'match', and reads the item directly from the node — no intermediate queue storage. The producer's `park()` is unparked by the consumer's CAS match. The item travels from producer's `SNode` directly to the consumer's local variable in $O(1)$. This avoids a copy into a buffer AND avoids the memory allocation of a queue node containing the item — only the `SNode` (not the item itself) is temporarily allocated.

[ADV] Q42. Explain DelayQueue's internal leader-follower pattern and why it avoids thundering herd on delay expiry.

DelayQueue uses a leader field (Thread reference). When `take()` finds the head element not yet expired, the first thread sets itself as leader and calls `Condition.awaitNanos(delay)` with a timed wait. All subsequent threads finding the queue non-empty but with unexpired head simply call `Condition.await()` (untimed) — they do NOT compute or wait for the specific expiry time. When the leader's timed wait expires, it acquires the lock, takes the element, and signals one follower to become the next leader. Without this pattern, all waiting threads would compute `getDelay()` and schedule timed waits simultaneously — causing a burst of wakeups at expiry time.

[ADV] Q43. Why is LinkedTransferQueue theoretically the best general BlockingQueue, combining SynchronousQueue and LinkedBlockingQueue capabilities?

LinkedTransferQueue merges two paradigms: (1) If a consumer is already waiting (`hasWaitingConsumer()`), `transfer()` delivers directly without buffering — same latency as `SynchronousQueue`. (2) If no consumer is waiting, the element is buffered in a lock-free linked list — same capacity as `LinkedBlockingQueue`. This means it never wastes a synchronous handoff opportunity AND never drops/blocks when no consumer is ready. Its lock-free implementation (CAS-based dual-data linked list) provides higher throughput than `LinkedBlockingQueue` under high contention. JDK 7+ makes it the preferred choice when you need both flexibility and performance.

[ADV] Q44. Explain ArrayDeque's circular buffer: how do head/tail wrap around using bitmasking? How does doubleCapacity() copy?

`ArrayDeque` always keeps `elements.length` as a power of 2 (e.g., 16, 32, 64). `mask = elements.length - 1` (e.g., 15 = 0b1111). New tail: `tail = (tail + 1) & mask` — equivalent to modulo but a single bitwise AND. New head: `head = (head - 1) & mask` — for `addFirst`. Full condition: `head == tail` (all slots occupied after wrap). `doubleCapacity()`: allocate `Object[2*n]`. Two `System.arraycopy` calls: (1) from head to end of old array (`n - head` elements), (2) from index 0 to head (`head` elements). The new array is laid out linearly: head becomes index 0, tail becomes index `n`.

[ADV] Q45. For top-K from a stream, why is a min-heap of size K optimal compared to a max-heap or sorting?

Min-heap of size `K` maintains the `K` largest seen so far. For each new element `x`: compare with heap's minimum (`peek = O(1)`). If `x > min`, `poll()` ($O(\log K)$) and `offer(x)` ($O(\log K)$). After `n` elements: $O(n \log K)$ time, $O(K)$ space. Why NOT max-heap? A max-heap would need to be of size `n` (all elements) to extract top-`K` — $O(n)$ space + $O(n + K \log n)$ time. Why NOT sorting? Sort all `n` elements: $O(n \log n)$ time, $O(n)$ space — worse for large `n` and small `K`. The min-heap approach is optimal because $K \ll n$ — $\log K$ is much smaller than $\log n$.

[ADV] Q46. How does a bounded BlockingQueue implement backpressure, and how does this compare to Reactive Streams' Subscriber.request(n)?

`BlockingQueue` backpressure: when full, `put()` blocks the producer thread — the OS parks the thread. This is implicit, push-based flow control. The producer cannot push faster than consumers can drain. Reactive Streams: `Subscriber.request(n)` is pull-based — the consumer explicitly signals how many items it can handle. The Publisher only emits up to `n` items. This avoids blocking threads entirely (non-blocking reactive pipelines). `BlockingQueue` is simpler but wastes thread resources. Reactive Streams has higher complexity but handles millions of events without blocking, using event loops (Netty/Reactor). For high-throughput async systems, Reactive Streams is superior.

[ADV] Q47. Explain ConcurrentLinkedQueue's Michael-Scott algorithm. Why does size() require O(n) traversal?

`ConcurrentLinkedQueue` uses the Michael-Scott non-blocking queue: a singly-linked list with CAS on the tail to append (`enqueue`) and CAS on the `head.next` to `dequeue`. No locks — compare-and-swap only. Linearizability: each operation appears to take effect at a single atomic CAS point. `size()` traversal: the queue does NOT maintain a counter (unlike `LinkedBlockingQueue`'s `AtomicInteger`). The reason: maintaining a counter atomically with CAS across head/tail modifications would require a two-pointer CAS (unavailable on most hardware). Instead, `size()` walks from head to tail counting nodes — $O(n)$ and only approximate (concurrent insertions/removals may happen during traversal).

[ADV] Q48. Compare PriorityQueue's heapify $O(n)$ with building a BST from n elements $O(n \log n)$. When does the heap's $O(n)$ construction matter?

Heap heapify (Floyd's): $O(n)$ amortized because most nodes are near the leaves and require only $O(1)$ sift-down. BST (balanced) insertion of n elements: $O(n \log n)$ — each insertion $O(\log n)$, n times. The $O(n)$ advantage matters when: (1) Building a heap from an existing collection once, then doing K extractions — $O(n + K \log n)$ vs repeated offer $O(n \log n)$. (2) Stream processing where you receive all data first then process in priority order. (3) Sorting (HeapSort): $O(n)$ build + $O(n \log n)$ extract = $O(n \log n)$ with better constant than comparison-based sorts.

[ADV] Q49. Explain how InterruptedException should be handled in BlockingQueue operations in production code.

InterruptedException signals that the calling thread has been interrupted (e.g., during shutdown). Two correct strategies: (1) Propagate: add 'throws InterruptedException' to calling method and let callers decide. Preferred in library code. (2) Restore interrupt flag: catch InterruptedException, call Thread.currentThread().interrupt() to re-set the flag, then return/cleanup. NEVER silently swallow: catch(InterruptedException e) {} — this masks shutdown signals and can prevent thread pools from terminating cleanly. In BlockingQueue consumer loops, interruption is the standard mechanism to signal thread pool shutdown.

[ADV] Q50. What is a dual-data structure in the context of LinkedTransferQueue, and how does it enable both buffering and synchronous handoff?

LinkedTransferQueue's internal linked list nodes can be in one of two modes: DATA (a producer deposited an item, waiting for a consumer) or REQUEST (a consumer is waiting for an item, holding a null item slot). When a producer calls offer() and a REQUEST node exists, the producer fills the node's item via CAS and unparks the consumer — synchronous handoff. If no REQUEST node, a DATA node is appended — buffered. Similarly for consumers: if a DATA node exists, consume it directly; else append a REQUEST node and park. This dual structure means the same linked list handles both buffered items and waiting threads, eliminating the need for separate data structures.

[ADV] Q51. How would you implement a bounded priority executor that gives high-priority tasks guaranteed slots in a bounded queue?

Use two queues: PriorityBlockingQueue highPriority (unbounded, only for high-priority tasks) + ArrayBlockingQueue normalQueue (bounded). Worker threads: poll highPriority first (non-blocking); if empty, poll normalQueue (with timeout). Producers: high-priority tasks go to highPriority; normal tasks go to normalQueue (with backpressure). Alternative: single PriorityBlockingQueue with a sentinel that gives high-priority tasks lower numeric priority values. The two-queue approach guarantees high-priority tasks are never delayed by normal queue backpressure.

[ADV] Q52. Describe the ABA problem in CAS-based queues and how Java's concurrent queues mitigate it.

ABA problem: thread T1 reads value A from a CAS location. Thread T2 changes A -> B -> A. T1's CAS succeeds even though the state changed. In queue context: a node dequeued and re-enqueued could cause T1 to incorrectly succeed on a stale pointer. ConcurrentLinkedQueue mitigates: nodes are never reused (fresh allocation per enqueue) — the ABA problem cannot occur because two allocations at the same address is impossible in Java (GC would have collected the intermediate state). SynchronousQueue mitigates similarly — nodes are created fresh per operation. Java's JVM memory model and GC make ABA far less problematic than in C/C++ where manual memory reuse is common.

Exercises — Part A: Coding Challenges

A1 — Bounded Priority Task Executor

Implement a task executor using `PriorityBlockingQueue` and a fixed thread pool. Tasks have integer priority (lower = higher priority). The executor should: (1) accept tasks with a priority, (2) execute in priority order, (3) support graceful shutdown with poison pills. Handle `InterruptedException` correctly.

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicBoolean;

class PriorityTask implements Comparable<PriorityTask> {
    final int priority;
    final Runnable task;
    PriorityTask(int p, Runnable t) { priority = p; task = t; }
    @Override public int compareTo(PriorityTask o) {
        return Integer.compare(this.priority, o.priority);
    }
}

class BoundedPriorityExecutor {
    private final PriorityBlockingQueue<PriorityTask> queue;
    private final ExecutorService pool;
    private final AtomicBoolean running = new AtomicBoolean(true);

    BoundedPriorityExecutor(int workers) {
        this.queue = new PriorityBlockingQueue<>(256);
        this.pool = Executors.newFixedThreadPool(workers);
        for (int i = 0; i < workers; i++) {
            pool.submit(() -> {
                while (running.get() || !queue.isEmpty()) {
                    try {
                        PriorityTask pt = queue.poll(100, TimeUnit.MILLISECONDS);
                        if (pt != null) pt.task.run();
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt(); break;
                    }
                }
            });
        }
    }

    void submit(int priority, Runnable task) {
        if (!running.get()) throw new IllegalStateException("Executor shut down");
        queue.offer(new PriorityTask(priority, task));
    }

    void shutdown() throws InterruptedException {
        running.set(false);
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
    }
}
```

A2 — Sliding Window Maximum (Monotonic Deque)

Given an array `nums` and window size `k`, return the maximum value in each sliding window. Use `ArrayDeque` as a monotonic deque storing indices. Achieve $O(n)$ time, $O(k)$ space.

```
public int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || nums.length == 0) return new int[0];
    int n = nums.length;
    int[] result = new int[n - k + 1];
```

```
Deque<Integer> deque = new ArrayDeque<>(); // stores indices
for (int i = 0; i < n; i++) {
    // Remove indices outside current window
    while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
        deque.pollFirst();
    }
    // Remove indices whose values are less than current
    // (they can never be the max for any future window)
    while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
        deque.pollLast();
    }
    deque.offerLast(i);
    // Start recording results once first window is filled
    if (i >= k - 1) {
        result[i - k + 1] = nums[deque.peekFirst()];
    }
}
return result;
}
// Time: O(n) – each index pushed/popped at most once
// Space: O(k) – deque holds at most k indices
```

A3 — Job Scheduler using DelayQueue

Implement a simple job scheduler where jobs can be scheduled to run after a specified delay. Support `submit(Runnable job, long delayMs)` and `shutdown()`. Use `DelayQueue`.

```
class ScheduledJob implements Delayed {
    private final Runnable task;
    private final long triggerTime; // nanoTime
    ScheduledJob(Runnable task, long delayMs) {
        this.task = task;
        this.triggerTime = System.nanoTime()
            + TimeUnit.MILLISECONDS.toNanos(delayMs);
    }
    @Override
    public long getDelay(TimeUnit unit) {
        return unit.convert(triggerTime - System.nanoTime(),
            TimeUnit.NANOSECONDS);
    }
    @Override
    public int compareTo(Delayed other) {
        return Long.compare(triggerTime, ((ScheduledJob) other).triggerTime);
    }
}

class SimpleJobScheduler {
    private final DelayQueue<ScheduledJob> queue = new DelayQueue<>();
    private final Thread dispatcher;
    private volatile boolean running = true;
    SimpleJobScheduler() {
        dispatcher = new Thread(() -> {
            while (running || !queue.isEmpty()) {
                try {
                    ScheduledJob job = queue.poll(100, TimeUnit.MILLISECONDS);
                    if (job != null) new Thread(job.task).start();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        });
    }
}
```



```

}
});
dispatcher.setDaemon(true);
dispatcher.start();
}

void submit(Runnable task, long delayMs) {
    queue.put(new ScheduledJob(task, delayMs));
}

void shutdown() {
    running = false;
    dispatcher.interrupt();
}
}
}

```

A4 — Work-Stealing Queue Simulation using ArrayDeque

Simulate a work-stealing thread pool where each worker has a local ArrayDeque. Workers process tasks from their own deque tail; steal from other workers' heads. Demonstrate the double-ended nature enabling local LIFO + remote FIFO access.

```

class WorkStealingPool {
    private final int numWorkers;
    private final Deque<Runnable>[] queues;
    private final Thread[] workers;
    private volatile boolean running = true;
    @SuppressWarnings("unchecked")
    WorkStealingPool(int n) {
        this.numWorkers = n;
        this.queues = new ArrayDeque[n];
        for (int i = 0; i < n; i++) queues[i] = new ArrayDeque<>();
        this.workers = new Thread[n];
        for (int i = 0; i < n; i++) {
            final int id = i;
            workers[i] = new Thread(() -> workerLoop(id));
            workers[i].start();
        }
    }

    void submit(int workerId, Runnable task) {
        synchronized (queues[workerId]) {
            queues[workerId].offerLast(task); // local push to tail
        }
    }

    private void workerLoop(int id) {
        while (running) {
            Runnable task = null;
            synchronized (queues[id]) {
                task = queues[id].pollLast(); // own tasks: LIFO from tail
            }
            if (task == null) {
                // Steal from another worker's head (FIFO)
                for (int i = 0; i < numWorkers; i++) {
                    if (i == id) continue;
                    synchronized (queues[i]) {
                        task = queues[i].pollFirst(); // steal from head
                    }
                    if (task != null) break;
                }
            }
        }
    }
}

```

```
if (task != null) task.run();
else Thread.yield();
}
}
void shutdown() { running = false; }
}
```

A5 — BFS on Graph using ArrayDeque

Implement breadth-first search on an unweighted graph using ArrayDeque as the queue. Return shortest distances from source to all reachable nodes. This is the standard interview pattern.

```
import java.util.*;

public int[] bfs(int[][] adjList, int n, int source) {
    int[] dist = new int[n];
    Arrays.fill(dist, -1);
    dist[source] = 0;

    Queue<Integer> queue = new ArrayDeque<>();
    queue.offer(source);

    while (!queue.isEmpty()) {
        int node = queue.poll();
        for (int neighbor : adjList[node]) {
            if (dist[neighbor] == -1) { // not visited
                dist[neighbor] = dist[node] + 1;
                queue.offer(neighbor);
            }
        }
    }

    return dist;
}

// Time: O(V + E), Space: O(V)
// ArrayDeque is preferred over LinkedList for BFS:
// - No node allocation overhead per enqueue
// - Better cache locality for large graphs
// - offer/poll are O(1) amortized
}
```

Part B — Debug & Fix

B1 — Stack Class Used for Recursion (Synchronized Overhead Bug)

```
Stack<Integer> callStack = new Stack<>();
callStack.push(1);
callStack.push(2);
int top = callStack.pop(); // correct, but...
// Stack extends Vector – every push/pop acquires a mutex!
// Also: callStack.get(0) works – should never be allowed on a stack!
```

Fix: Stack extends Vector, making every operation synchronized. Even in single-threaded code, each push/pop acquires and releases a monitor lock — unnecessary overhead. Worse, Stack exposes all List/Vector methods (add, get, set, remove by index) — allowing arbitrary access that violates the stack contract. Fix: use Deque callStack = new ArrayDeque<>(); push() = addFirst(), pop() = removeFirst(), peek() = peekFirst(). No synchronization, proper encapsulation.

B2 — Assuming PriorityQueue Iterator Returns Sorted Order

```
PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 3, 2, 4));
for (int x : pq) {
    System.out.print(x + " "); // prints: 1 2 3 5 4 (heap order, NOT sorted!)
}
// BUG: developer assumed iteration = sorted output
```

Fix: PriorityQueue's iterator traverses the internal array in index order (0, 1, 2, ...). The array satisfies heap property (parent <= children) but is NOT fully sorted. Only index 0 (the root) is guaranteed to be the minimum. Fix: to consume in sorted order, use a loop: while (!pq.isEmpty()) System.out.print(pq.poll()); — each poll() removes and returns the current minimum in O(log n). Or: convert to List, sort the list (O(n log n)).

B3 — Calling take() on a Non-BlockingQueue Substitute Leading to Infinite Wait

```
Queue<Task> queue = new LinkedList<>();
// Consumer thread:
while (true) {
    Task t = queue.poll(); // returns null if empty – no blocking!
    if (t != null) process(t);
    // BUG: busy-wait loop! Wastes CPU; no blocking sleep
    // If producers stop, consumer spins forever on empty queue
}
```

Fix: LinkedList.poll() is non-blocking — it returns null immediately if empty. The consumer loop becomes a busy-wait, consuming 100% CPU with no useful work. Fix: replace LinkedList with a BlockingQueue and call take() which parks the thread when empty: BlockingQueue queue = new LinkedBlockingQueue<>(500); Task t = queue.take(); // blocks efficiently until a task arrives. Always use BlockingQueue for producer-consumer — never poll() in a busy loop.

B4 — Deadlock from Missing Poison Pill in Multi-Producer Setup

```
BlockingQueue<Task> queue = new LinkedBlockingQueue<>(100);  
// 3 producers, 2 consumers  
// Producer shutdown: each producer puts ONE poison pill  
queue.put(POISON_PILL); // from producer 1  
queue.put(POISON_PILL); // from producer 2  
queue.put(POISON_PILL); // from producer 3  
// BUG: 3 pills for 2 consumers – one consumer gets 2 pills  
// and terminates early; other consumer hangs forever on take()  
// if producers finish but only 2 pills delivered to consumers
```

Fix: The number of poison pills must equal the number of consumers, not producers. If each producer independently puts one pill, the total pills = num_producers. If num_producers != num_consumers, some consumers get no pill and hang on take() forever. Fix: have exactly ONE designated cleanup step that puts num_consumers poison pills: for (int i = 0; i < NUM_CONSUMERS; i++) queue.put(POISON_PILL); This should happen after ALL producers have finished, using a CountdownLatch or CompletableFuture.allOf() to coordinate. Alternatively, use a volatile shutdown flag + thread interruption instead of poison pills for more robust shutdown.

Part C — MCQ / Predict the Output

MC1. What is the output? Deque d = new ArrayDeque<>(); d.push(1); d.push(2); d.push(3); System.out.println(d.poll());

- A) 1
- B) 2
- C) 3
- D) null

Answer: C — push() calls addFirst(). So deque = [3, 2, 1] (3 at front). poll() calls removeFirst(), returning 3.

MC2. What is the output? PriorityQueue pq = new PriorityQueue<>(); pq.offer(5); pq.offer(1); pq.offer(3); System.out.println(pq.peek() + " " + pq.poll());

- A) 5 5
- B) 1 1
- C) 5 1
- D) 1 5

Answer: B — PriorityQueue is a min-heap. peek() returns 1 (minimum) without removing. poll() also returns 1 (the minimum). Output: '1 1'.

MC3. Which BlockingQueue allows simultaneous put() and take() from different threads without contention?

- A) ArrayBlockingQueue
- B) SynchronousQueue
- C) LinkedBlockingQueue
- D) PriorityBlockingQueue

Answer: C — LinkedBlockingQueue uses separate putLock and takeLock — a producer and consumer can hold different locks simultaneously, avoiding contention. ArrayBlockingQueue and PriorityBlockingQueue use a single lock; SynchronousQueue has no storage at all.

MC4. What happens when you call put() on a SynchronousQueue and no consumer is waiting?

- A) Element is buffered and put() returns
- B) IllegalStateException is thrown
- C) put() blocks until a consumer calls take()
- D) put() returns false

Answer: C — SynchronousQueue has zero capacity. put() always blocks until a consumer calls take() — a direct rendezvous. offer() (non-blocking) would return false immediately if no consumer is waiting.

MC5. PriorityQueue pq = new PriorityQueue<>(Comparator.reverseOrder()); pq.offer(3); pq.offer(1); pq.offer(5); System.out.println(pq.poll());

- A) 1
- B) 3
- C) 5
- D) Compile error

Answer: C — Comparator.reverseOrder() creates a max-heap. The maximum element (5) is at the head. poll() removes and returns 5.

MC6. Which statement about ArrayDeque is FALSE?

- A) It allows null elements
- B) It is faster than Stack for LIFO
- C) It implements Deque
- D) It resizes by doubling capacity

Answer: A — ArrayDeque does NOT allow null elements — null is used internally as a sentinel to detect empty slots in the circular buffer. Statements B, C, D are all true.

Quick Reference Cheat Sheet

Topic	Key Points to Remember
Queue method pairs	offer/add (false vs exception); poll/remove (null vs exception); peek/element (null vs exception)
PriorityQueue internals	Min-heap Object[] array; $O(\log n)$ offer/poll; $O(1)$ peek; $O(n)$ contains/remove; heapify $O(n)$
PriorityQueue iterator	NOT sorted — traverses array in index order; use repeated poll() for sorted output
Max-heap	new PriorityQueue<>(Comparator.reverseOrder()) — reverses natural ordering to max-heap
Top-K pattern	Min-heap of size K; poll when size > K; $O(n \log K)$ time, $O(K)$ space — optimal
ArrayDeque	Circular array; power-of-2 capacity; $O(1)$ both ends; no null; prefer over Stack and LinkedList
Stack with Deque	push=addFirst, pop=removeFirst, peek=peekFirst; never use legacy Stack class
Queue with Deque	offer=addLast, poll=removeFirst, peek=peekFirst
LinkedBlockingQueue	Dual-lock (putLock + takeLock); concurrent put+take; default capacity = MAX_VALUE
ArrayBlockingQueue	Single lock; mandatory capacity; optional fairness (reduces throughput)
SynchronousQueue	Zero storage; direct handoff; CAS-based; used in newCachedThreadPool()
DelayQueue	Elements implement Delayed; take() waits for expiry; leader-follower avoids thundering herd
PriorityBlockingQueue	Unbounded; put() never blocks; take() blocks if empty; thread-safe priority queue
LinkedTransferQueue	Best general BlockingQueue; direct handoff if consumer waiting, else buffer; lock-free
Poison pill shutdown	Put NUM_CONSUMERS pills AFTER all producers finish; not NUM_PRODUCERS pills
Backpressure	Bounded BlockingQueue: put() blocks producer when full — natural flow control
drainTo()	Atomically removes all/up to n elements; more efficient than looping poll()
ConcurrentLinkedQueue	Non-blocking (CAS); no size() in $O(1)$; use when threads are always active

End of Sub-Chapter 1.5 — Collections: Queue, Deque & BlockingQueue

Next: Sub-Chapter 1.6 — Collections Internals, Algorithms & Best Practices